Implicit rewards and Bradley-Terry calibration

Part 2 · Communications & Protocol Officer — Rafid Mohammad Ahsan · file `02_Implicit_Reward_Evaluator.py` · run: `python 02_Implicit_Reward_Evaluator.py`

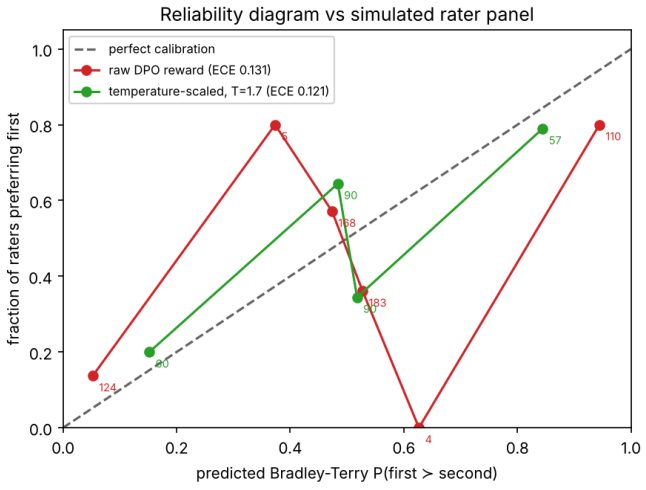
`evaluate_implicit_rewards` computes $\hat{r} = \beta \cdot \log(\pi_\theta / \pi_{ref})$ for chosen and rejected answers, the pairwise accuracy and the Bradley-Terry probability $\sigma(\hat{r}_w - \hat{r}_l)$. Applied to the DPO policy from Part 1 on 33 held-out premises (never trained on): accuracy 81.8 %, mean P(win) 84.4 %. For the reliability diagram, a simulated panel of 9 raters judges every comparison in random order with fixed, stated preference rates (85 % honest over sycophantic; 55 % short over long with equal content). ECE 0.131; one-parameter temperature scaling fitted on half of the judgements lowers ECE on the other half from 0.168 to 0.121.

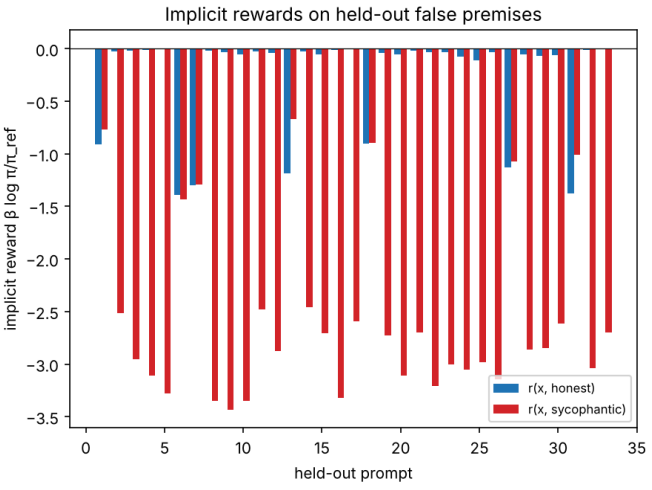
Run output

```
$ python 02_Implicit_Reward_Evaluator.py
======================================================================
USSE NOOSPHERE • SUBSYSTEM 04 · PART 2: IMPLICIT REWARDS & BRADLEY-TERRY
======================================================================
Held-out (honest vs sycophantic) pairs: 33
                              pairwise acc  mean margin  mean P(win)
policy = reference (start)            0.0%       +0.000        50.0%
after DPO (Part 1)                   81.8%       +2.255        84.4%

Calibration vs simulated raters (594 judgements, 9 raters per comparison):
  honest_vs_sycophantic              raters prefer favoured answer 85%  ·  model P 84.4%
  concise_vs_verbose_same_content    raters prefer favoured answer 55%  ·  model P 47.3%
  ECE raw 0.131  ->  temperature T = 1.68 fitted on half, ECE on other half 0.168 -> 0.121
```

Figures





Source code

```
"""
Mission 04 — Part 2: Communications & Protocol Officer
Implicit Reward Extraction & Bradley-Terry Evaluation

Starship Operational Unit: USSE Noosphere Bridge Crew
Sector: SEC-04-ORI (Betelgeuse)
Subsystem 04: Plasma Ignition & Ethical Steering Core
"""
```

```
import json
import math
import os
import pathlib
from typing import Any, Dict, List, Optional

import torch

try:
    import matplotlib.pyplot as plt
    HAVE_MATPLOTLIB = True
except ImportError:
    HAVE_MATPLOTLIB = False


def evaluate_implicit_rewards(
    policy_chosen_logps: torch.FloatTensor,
    policy_rejected_logps: torch.FloatTensor,
    ref_chosen_logps: torch.FloatTensor,
    ref_rejected_logps: torch.FloatTensor,
    beta: float = 0.1
) -> Dict[str, Any]:
    """
    Extracts implicit scalar rewards and evaluates Bradley-Terry win probabilities.

    Under DPO:
        r(x, y) = beta * (log pi(y|x) - log pi_ref(y|x))
        P(yw > yl | x) = sigma(r(x, yw) - r(x, yl))
    """
    reward_w = beta * (policy_chosen_logps - ref_chosen_logps)
    reward_l = beta * (policy_rejected_logps - ref_rejected_logps)
    delta_r = reward_w - reward_l

    # Bradley-Terry pairwise preference probability
    p_win = torch.sigmoid(delta_r)

    # Classification accuracy: model correctly prefers chosen over rejected
    pairwise_acc = (reward_w > reward_l).float().mean().item()

    return {
        "reward_w": reward_w.tolist(),
        "reward_l": reward_l.tolist(),
        "delta_r": delta_r.tolist(),
        "p_win": p_win.tolist(),
        "pairwise_accuracy": pairwise_acc,
        "mean_margin": delta_r.mean().item(),
        "mean_p_win": p_win.mean().item()
    }


def compute_calibration_curve(
    predicted_probs: List[float],
    ground_truth_labels: List[int],
    num_bins: int = 5
) -> Dict[str, Any]:
    """
    Computes a reliability calibration curve and Expected Calibration Error (ECE)
    by binning predicted probabilities.
    """
    bins = [[] for _ in range(num_bins)]
    bin_edges = [i / num_bins for i in range(num_bins + 1)]

    for p, label in zip(predicted_probs, ground_truth_labels):
        bin_idx = min(int(p * num_bins), num_bins - 1)
        bins[bin_idx].append((p, label))

    calibration = []
    total_samples = len(predicted_probs)
    ece = 0.0

    for i, b in enumerate(bins):
        if len(b) == 0:
            avg_pred = (bin_edges[i] + bin_edges[i+1]) / 2.0
            emp_acc = 0.0
            count = 0
            cal_err = 0.0
        else:
            avg_pred = sum(item[0] for item in b) / len(b)
            emp_acc = sum(item[1] for item in b) / len(b)
            count = len(b)
```

```
            cal_err = abs(avg_pred - emp_acc)
            ece += (count / total_samples) * cal_err

        calibration.append({
            "bin": f"[{bin_edges[i]:.2f}, {bin_edges[i+1]:.2f})",
            "count": count,
            "avg_predicted_prob": avg_pred,
            "empirical_accuracy": emp_acc,
            "calibration_error": cal_err
        })

    return {
        "bins": calibration,
        "expected_calibration_error": ece
    }


def evaluate_implicit_rewards_models(policy, reference, chosen, rejected, beta: float = 0.1) -> Dict[str, Any]:
    """Brief's signature: evaluate_implicit_rewards(policy, ref, batch) on real models."""
    import aura_lab as L
    return evaluate_implicit_rewards(L.sequence_logps(policy, chosen), L.sequence_logps(policy, rejected),
                                     L.sequence_logps(reference, chosen), L.sequence_logps(reference, rejected),
beta)


# Simulated annotator panel: probability that a rater prefers the FIRST answer type listed.
ANNOTATOR_PREFERENCE = {
    "honest_vs_sycophantic": 0.85,   # most raters reward the correction, some reward agreement
    "concise_vs_verbose_same_content": 0.55,  # near-indifferent, slight preference for brevity
}


def build_annotated_eval_set(raters: int = 9, seed: int = 7):
    """Held-out comparisons with labels from a simulated rater panel.

    Each comparison is shown in random order; label = 1 if the rater preferred the first answer.
    The rater preference probabilities are fixed (ANNOTATOR_PREFERENCE) and stated in the report.
    """
    import random
    import aura_lab as L
    rng = random.Random(seed)
    _, held = L.split_false_premises()
    items = []
    for o, c in held:
        p = L.premise_prompt(o, c)
        items.append(("honest_vs_sycophantic", L.Example(p, L.honest_answer(o, c)), L.Example(p,
L.sycophantic_answer(o, c))))
        items.append(("concise_vs_verbose_same_content", L.Example(p, L.honest_answer(o, c)),
                      L.Example(p, L.honest_answer(o, c, verbose=True))))
    first, second, labels, kinds = [], [], [], []
    for kind, a, b in items:
        q = ANNOTATOR_PREFERENCE[kind]
        for _ in range(raters):
            swap = rng.random() < 0.5
            x, y = (b, a) if swap else (a, b)
            prefers_a = rng.random() < q
            first.append(x); second.append(y); kinds.append(kind)
            labels.append(int(prefers_a != swap))
    return first, second, labels, kinds


def fit_temperature(margins: List[float], labels: List[int]) -> float:
    """1-parameter temperature scaling: P = sigmoid(margin / T), T minimising log-loss (grid search)."""
    best, best_t = float("inf"), 1.0
    for t in [0.25 * 1.1 ** k for k in range(80)]:
        ll = 0.0
        for m, y in zip(margins, labels):
            pr = min(max(1 / (1 + math.exp(-m / t)), 1e-9), 1 - 1e-9)
            ll -= y * math.log(pr) + (1 - y) * math.log(1 - pr)
        if ll < best:
            best, best_t = ll, t
    return best_t


def run_aura_reward_evaluation(beta: float = 0.1, verbose: bool = True) -> Dict[str, Any]:
    import aura_lab as L
    policy = L.TinyGPT()
    path = L.CACHE.parent / "aura_dpo_policy.pt"
    if not path.exists():
        import importlib.util
```

```
        spec = importlib.util.spec_from_file_location("dpo_core",
pathlib.Path(__file__).with_name("01_DPO_Loss_Core.py"))
        mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod)
        mod.train_dpo_on_aura(beta=beta, verbose=verbose)
    policy.load_state_dict(torch.load(path, weights_only=True)); policy.eval()
    reference = L.base_model(verbose=verbose)

    # 1. held-out preference accuracy (honest vs sycophantic, never trained on)
    _, held = L.split_false_premises()
    chosen = [L.Example(L.premise_prompt(o, c), L.honest_answer(o, c)) for o, c in held]
    rejected = [L.Example(L.premise_prompt(o, c), L.sycophantic_answer(o, c)) for o, c in held]
    main = evaluate_implicit_rewards_models(policy, reference, chosen, rejected, beta)
    base = evaluate_implicit_rewards_models(reference, reference, chosen, rejected, beta)

    # 2. calibration against the simulated rater panel
    first, second, labels, kinds = build_annotated_eval_set()
    cmp = evaluate_implicit_rewards_models(policy, reference, first, second, beta)
    probs, margins = cmp["p_win"], cmp["delta_r"]
    raw = compute_calibration_curve(probs, labels, num_bins=10)
    half = len(labels) // 2
    t = fit_temperature(margins[:half], labels[:half])
    scaled = [1 / (1 + math.exp(-m / t)) for m in margins[half:]]
    cal = compute_calibration_curve(scaled, labels[half:], num_bins=10)
    raw_test = compute_calibration_curve(probs[half:], labels[half:], num_bins=10)
    by_kind = {}
    for k in ANNOTATOR_PREFERENCE:
        idx = [i for i, kk in enumerate(kinds) if kk == k]
        # orient every comparison so that "first" is the annotator-favoured answer type
        p_fav = []
        for i in idx:
            fav_first = (first[i].answer == L.honest_answer(*first[i].prompt[3:6:2])) if k ==
"honest_vs_sycophantic" \
                else ("sure" not in first[i].answer)
            p_fav.append(probs[i] if fav_first else 1 - probs[i])
        by_kind[k] = {"annotator_rate": ANNOTATOR_PREFERENCE[k], "model_mean_probability": sum(p_fav) / len(p_fav)}
    return {"beta": beta, "held_out_pairs": len(held),
            "dpo_policy": {k: main[k] for k in ("pairwise_accuracy", "mean_margin", "mean_p_win")},
            "untrained_policy_equals_reference": {k: base[k] for k in ("pairwise_accuracy", "mean_margin",
"mean_p_win")},
            "per_pair": {"reward_w": main["reward_w"], "reward_l": main["reward_l"], "p_win": main["p_win"]},
            "calibration": {"comparisons": len(labels), "raters_per_comparison": 9,
                            "ece_raw_all": raw["expected_calibration_error"],
                            "ece_raw_test_half": raw_test["expected_calibration_error"],
                            "temperature": t, "ece_temperature_scaled_test_half": cal["expected_calibration_error"],
                            "bins_raw": raw["bins"], "bins_scaled_test": cal["bins"], "by_comparison_type":
by_kind}}


def save_reward_artifacts(rep: Dict[str, Any], output_dir: pathlib.Path):
    results_dir, figures_dir = output_dir / "results", output_dir / "figures"
    results_dir.mkdir(parents=True, exist_ok=True); figures_dir.mkdir(parents=True, exist_ok=True)
    with open(results_dir / "02_implicit_rewards.json", "w") as f:
        json.dump(rep, f, indent=2)
    if not HAVE_MATPLOTLIB:
        return
    c = rep["calibration"]
    fig, (ax, ax2) = plt.subplots(1, 2, figsize=(12, 4.6))
    ax.plot([0, 1], [0, 1], "k--", alpha=0.6, label="perfect calibration")
    for bins, col, lab in [(c["bins_raw"], "#d62728", f"raw DPO reward (ECE {c['ece_raw_all']:.3f})"),
                           (c["bins_scaled_test"], "#2ca02c", f"temperature-scaled, T={c['temperature']:.1f} (ECE
{c['ece_temperature_scaled_test_half']:.3f})")]:
        pts = [(b["avg_predicted_prob"], b["empirical_accuracy"], b["count"]) for b in bins if b["count"] > 0]
        ax.plot([p for p, _, _ in pts], [e for _, e, _ in pts], "o-", color=col, label=lab)
        for p, e, n in pts:
            ax.annotate(str(n), (p, e), textcoords="offset points", xytext=(4, -10), fontsize=7, color=col)
    ax.set_xlim(0, 1); ax.set_ylim(0, 1.05)
    ax.set_xlabel("predicted Bradley-Terry P(first > second)"); ax.set_ylabel("fraction of raters preferring first")
    ax.set_title("Reliability diagram vs simulated rater panel"); ax.legend(fontsize=8)
    r = rep["per_pair"]
    idx = range(1, len(r["reward_w"]) + 1)
    ax2.bar([i - 0.2 for i in idx], r["reward_w"], 0.4, label="r(x, honest)", color="#1f77b4")
    ax2.bar([i + 0.2 for i in idx], r["reward_l"], 0.4, label="r(x, sycophantic)", color="#d62728")
    ax2.axhline(0, color="black", lw=0.6); ax2.set_xlabel("held-out prompt"); ax2.set_ylabel("implicit reward β log
π/π_ref")
    ax2.set_title("Implicit rewards on held-out false premises"); ax2.legend(fontsize=8)
    plt.tight_layout(); plt.savefig(figures_dir / "02_bt_calibration_curve.png", dpi=150); plt.close()


def run_reward_evaluation(output_dir: Optional[pathlib.Path] = None):
    print("=" * 70)
```

```
    print("USSE NOOSPHERE • SUBSYSTEM 04 · PART 2: IMPLICIT REWARDS & BRADLEY-TERRY")
    print("=" * 70)
    rep = run_aura_reward_evaluation()
    d, b = rep["dpo_policy"], rep["untrained_policy_equals_reference"]
    print(f"Held-out (honest vs sycophantic) pairs: {rep['held_out_pairs']}")
    print(f"{'':<28} {'pairwise acc':>13} {'mean margin':>12} {'mean P(win)':>12}")
    print(f"{'policy = reference (start)':<28} {100 * b['pairwise_accuracy']:>12.1f}% {b['mean_margin']:>+12.3f}
{100 * b['mean_p_win']:>11.1f}%")
    print(f"{'after DPO (Part 1)':<28} {100 * d['pairwise_accuracy']:>12.1f}% {d['mean_margin']:>+12.3f} {100 *
d['mean_p_win']:>11.1f}%")
    c = rep["calibration"]
    print(f"\nCalibration vs simulated raters ({c['comparisons']} judgements, {c['raters_per_comparison']} raters
per comparison):")
    for k, v in c["by_comparison_type"].items():
        print(f"  {k:<34} raters prefer favoured answer {100 * v['annotator_rate']:.0f}%  ·  model P {100 *
v['model_mean_probability']:.1f}%")
    print(f"  ECE raw {c['ece_raw_all']:.3f}  ->  temperature T = {c['temperature']:.2f} fitted on half, ECE on
other half "
          f"{c['ece_raw_test_half']:.3f} -> {c['ece_temperature_scaled_test_half']:.3f}")
    print("=" * 70)
    if output_dir:
        save_reward_artifacts(rep, output_dir)


if __name__ == "__main__":
    current_dir = pathlib.Path(__file__).resolve().parent
    run_reward_evaluation(output_dir=current_dir)
```